

ArtBench-10 Generative Modeling: VAE, DCGAN, and Diffusion

Nuno Batista and Miguel Cabral Pinto

University of Coimbra

1 Introduction

We study generative modeling on ArtBench-10, a 10-class dataset of 32×32 paintings [6]. We implement and compare a VAE [5], a DCGAN and conditional variant (cGAN) [8], and diffusion-based models [4], including fine-tuning a pre-trained Google DDPM. Models are selected via Bayesian hyperparameter search and sample quality is evaluated with FID [3], KID [1], and Inception Score (IS) [9]. We report results across 10 random seeds.

2 Problem Description

The target domain is ArtBench-10 [6], a curated benchmark of 50,000 training and 10,000 test images across 10 artistic styles. Each image is a 32×32 RGB painting. The objective is to learn a generative model $p_\theta(x)$ that can sample plausible paintings while maintaining fidelity and diversity.

3 Methodology and Approach

3.1 Data Pipeline

The data pipeline handles the ArtBench-10 dataset (32×32 RGB images, 10 styles). Images are converted from integer pixels to floating point and normalized to the range $[-1, 1]$, which centers the data around zero and standardizes the range across models, which aligns the data scale with common generative-model output parameterizations (e.g., a `tanh` output layer naturally produces values in $[-1, 1]$). For diffusion models, using a symmetric, zero-centered range is also convenient because the forward noising process is defined around zero-mean Gaussian noise.

During the experimentation phase, a 20% subset (10,000 images) was used for hyperparameter sweeps via Bayesian optimization. Once optimal parameters were identified, the final models were retrained on the full 50,000-image training set.

3.2 Model Families

The architectural design choices for each model family were tailored to the 32×32 resolution.

- **VAE:** Our VAE features a symmetric 4-layer convolutional encoder–decoder ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$). The encoder applies strided convolutions down to a spatial bottleneck with parameterizable dimensions. The decoder mirrors this with transposed convolutions. Each layer uses Batch Normalization and LeakyReLU (slope 0.2). The reparameterization trick enables end-to-end backpropagation through the stochastic bottleneck.
- **DCGAN and cGAN:** Both GAN families follow the DCGAN blueprint [8]: 4-layer CNNs with strided-convolution downsampling, transposed-convolution upsampling, Batch Normalization, ReLU activations in the generator, and LeakyReLU in the discriminator, compressing to a $1 \times 1 \times 64$ noise bottleneck. The cGAN variant adds a style embedding concatenated with the noise vector in the generator, and injects the same embedding as an extra channel in the discriminator for class-conditional generation.
- **Diffusion (From Scratch):** Our custom UNet uses two downsampling and two upsampling stages built from Residual Blocks with Group Normalization and SiLU (Swish) activations, reaching an $8 \times 8 \times 512$ bottleneck via strided convolutions and recovering spatial resolution with transposed convolutions. Conditioning is applied through sinusoidal time-step embeddings combined with class embeddings, and optional bottleneck self-attention can be enabled. Skip connections preserve high-frequency stylistic details.
- **Diffusion (Pretrained Google DDPM):** The `google/ddpm-cifar10-32` checkpoint [2] was fine-tuned on ArtBench. It uses a deeper 4-stage UNet with Upsample+Conv upsampling, strided-convolution downsampling, Group Normalization, SiLU activations, a $4 \times 4 \times 512$ bottleneck, multi-head self-attention at multiple resolutions, and sinusoidal timestep conditioning.

3.3 Implementation and Reproducibility

Our pipeline is configuration-driven via YAML files. We utilize Weights & Biases (W&B) for real-time metric logging. All experiments were conducted using PyTorch, with random seeds fixed to 67 for hyperparameter comparisons.

3.4 Analysis of Hyperparameters

We summarize the swept hyperparameters and their primary effects on optimization stability, capacity, and sample quality. In this analysis, we focus on the most relevant and specific hyperparameters for each model family.

Variational Autoencoders (VAE)

- **Learning rate (lr):** Controls optimization step size; too large can destabilize training and degrade reconstructions, while too small slows convergence.
- **Latent dimension (latent_dim):** Sets bottleneck capacity; larger values increase expressiveness but can weaken regularization and encourage blurry reconstructions if the decoder dominates.

- **KL weight (β):** Scales the KL regularizer in the VAE objective. With a reconstruction loss term and a KL term, increasing β reduces the information capacity of the latent (it discourages encoding fine detail in z), typically improving sampling from the prior and latent smoothness but worsening reconstruction fidelity. Decreasing β does the opposite: it behaves more like a deterministic autoencoder ($\beta \rightarrow 0$), often improving reconstructions but producing a latent distribution that deviates from the prior, which can harm unconditional sample quality when sampling $z \sim p(z)$.

Generative Adversarial Networks (GAN)

- **Learning rate (lr):** One of the most sensitive GAN hyperparameters. Higher values can cause divergence or oscillations in the discriminator and generator game.
- **Latent dimension (latent_dim):** Controls the dimensionality of the generator input noise. Too small can limit diversity, while too large can make mapping harder without improving quality.
- **Base channels (base_channels):** Changes generator/discriminator capacity. Imbalance can lead to either an overpowered discriminator (vanishing gradients) or an underpowered discriminator (poor signal).
- **Discriminator updates (d_updates_per_g):** Sets the ratio of discriminator steps per generator step. Increasing it can stabilize training when the discriminator is weak, but can also over-train the discriminator and stall generator learning.
- **Spectral normalization (use_spectral_norm):** Enforces Lipschitzness, a mathematical cap on how rapidly the discriminator’s output can change between similar inputs. This makes it so the discriminator can’t output completely different scores for two very similar images. This constraint prevents extreme gradients and improves training stability. However, forcing this smoothness reduces the discriminator’s flexibility to learn sharp, complex boundaries, potentially slowing down learning.

Conditional GAN (cGAN)

- **Generator conditioning (concat in latent space):** Given a noise vector z (dimension d) and a class label y , the generator embeds the label with an embedding layer $e(y)$ (dimension 64) and concatenates them $[z; e(y)]$. The concatenated vector is reshaped to $(d + 64) \times 1 \times 1$ and fed through the standard DCGAN transposed-convolution stack. This makes the class signal available from the first layer onward.
- **Discriminator conditioning (label as an image channel):** The discriminator embeds y into a vector of size $32 \cdot 32$, reshapes it into a $1 \times 32 \times 32$

map, and concatenates it with the RGB image as an extra input channel (input becomes $4 \times 32 \times 32$). The discriminator therefore learns to score joint pairs (x, y) rather than marginal realism of x .

- **Effect on optimization:** Conditioning increases discriminator capacity (it can use the label map as a shortcut feature). If the discriminator learns class-specific cues faster than the generator can match them, gradients to the generator become less informative, reducing both diversity and conditional fidelity.
- **Mismatch loss weight (λ):** The cGAN discriminator is trained with an additional mismatch penalty: real images paired with the *wrong* class label are treated as fake. λ controls how strongly this term is weighted relative to the standard adversarial loss. A low λ lets the discriminator ignore label consistency, reducing the pressure on the generator to produce class-specific outputs. This was held fixed at $\lambda = 0.2$ during the sweep and varied in the mode collapse study (Section 5.3).

Diffusion Models

- **Base channels (`base_channels`):** Sets UNet width. Larger models typically improve sample fidelity and style capture at increased compute.
- **Attention (`use_attention`):** When enabled, the UNet inserts a multi-head self-attention block at the bottleneck feature map (after the mid residual block, at the lowest spatial resolution). Concretely, it computes Q, K, V via a 1×1 convolution, flattens the $H \times W$ spatial grid into tokens, and applies scaled dot-product attention so each location can aggregate information from all other locations. The attention output is projected back with a 1×1 convolution and added residually. This can improve global coherence (e.g., consistent structure across distant regions) at relatively low cost because the bottleneck has the smallest $H \times W$.
- **Learning Rate on Fine-tuning:** For Google DDPM, a lower learning rate improved adaptation from CIFAR-10 priors to ArtBench without leading to catastrophic divergence.

4 Experimental Setup

4.1 Hyperparameter Search

Hyperparameter selection was performed on a **20% training subset** (10,000 images) using **Bayesian optimization**. Each sweep run trained a candidate configuration and evaluated sample quality using **FID**. For computationally expensive diffusion-based models, sweep runs used reduced evaluation settings (e.g., fewer generated samples and shorter training) and the best configuration was then re-trained with the final evaluation settings. Our stopping criteria, though varying

across models, was around the point where the best FID plateaued for a significant number of consecutive runs. Search spaces and budgets were model-specific, as seen in Table 1:

Table 1. Hyperparameter search spaces for each model’s Bayesian sweep

Model	Parameter	Values
VAE	lr	{0.00005, 0.0001, 0.0005, 0.001, 0.002}
	latent_dim	{64, 128, 256}
	base_channels	{64, 128}
	β	{0.5, 1, 2, 4}
	batch_size	{64, 128, 256}
DCGAN / cGAN	lr	{0.00005, 0.0002, 0.001}
	latent_dim	{64, 100, 128}
	base_channels	{64, 128}
	d_updates_per_g	{1, 2, 5}
	use_spectral_norm	{true, false}
Diffusion	lr	{0.00005, 0.0002, 0.0005}
	base_channels	{64, 128}
	use_attention	{true, false}
Google DDPM	lr	{0.00001, 0.00005, 0.0002, 0.0005, 0.01}
	epochs	{10, 20, 40}

To visualize the explored hyperparameter configurations and their performance, we used Weights & Biases sweep diagnostics. We used **fANOVA importance analysis** to estimate which parameters most influenced FID (Figure 1).

Several patterns emerge from the sweeps. Learning rate is the dominant factor for most models. An exception is the **Diffusion model**, where the binary `use_attention` flag alone explains 96.4% of variance, all low-FID runs have `use_attention=false`. At 32×32 resolution, the additional complexity of self-attention appears to impair rather than help optimization. For the **VAE**, the KL weight **beta** is the second most important parameter, which is expected given its direct control over the reconstruction-regularization trade-off. the sweep selected a low value ($\beta = 0.5$), prioritizing reconstruction fidelity. The **DCGAN** sweep found `use_spectral_norm` nearly irrelevant (0.2%), suggesting that at this scale the standard adversarial objective is already stable enough. In contrast, the **cGAN** importance profile is remarkably flat across all five parameters (range 0.105-0.288). For the **Google DDPM**, the lower learning rates and more training epochs consistently yield lower FID, with `lr=0.01` leading to catastrophic divergence (FID ≈ 500).

Table 2 shows the configuration with the lowest FID obtained for each model.

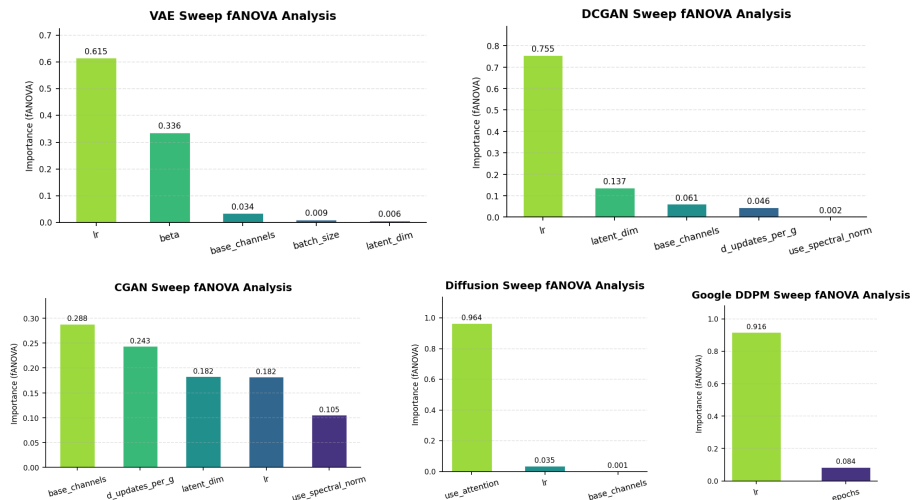


Fig. 1. fANOVA parameter importance for the hyperparameter sweeps, reporting the estimated contribution of each hyperparameter to FID variance for each model family.

Table 2. Final hyperparameter configurations for full-dataset training.

Parameter	VAE	DCGAN	CGAN	Diffusion	Google DDPM
Learning Rate	0.002	0.0002	0.00005	0.0002	0.0005
Batch Size	256	128	128	32	32
Latent Dim.	256	64	128	128	128
Base Channels	128	128	128	128	128
Optimizer	Adam	Adam	Adam	AdamW	AdamW
Beta (β)	0.5	-	-	-	-
D. Updates per G.	-	1	5	-	-
Spectral Norm	-	No	No	-	-
Attention	-	-	-	No	-

4.2 Experimental Protocol

After each sweep, we used the hyperparameters in Table 2 to train the models on the 20% subset. We then performed a **final evaluation** by training that best configuration and computing FID/KID using **5,000 generated** samples versus **5,000 real** samples (KID computed over 50 random subsets of size 100), with the evaluation repeated across **10 different random seeds** for statistical robustness (we report Mean $\pm$ Standard Deviation). Finally, we retrained all model families on the **full 50k** training set using their best hyperparameters and repeated the same 10-seed evaluation.

5 Results and Analysis

5.1 Qualitative Results

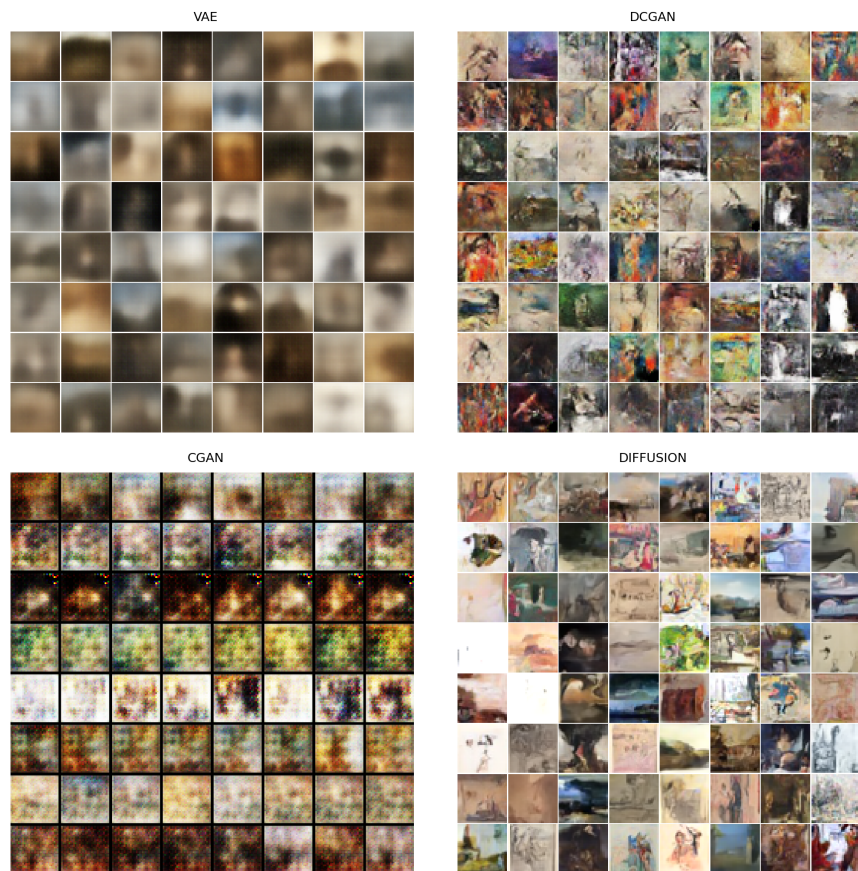


Fig. 2. Qualitative samples from the main model families evaluated on ArtBench-10.

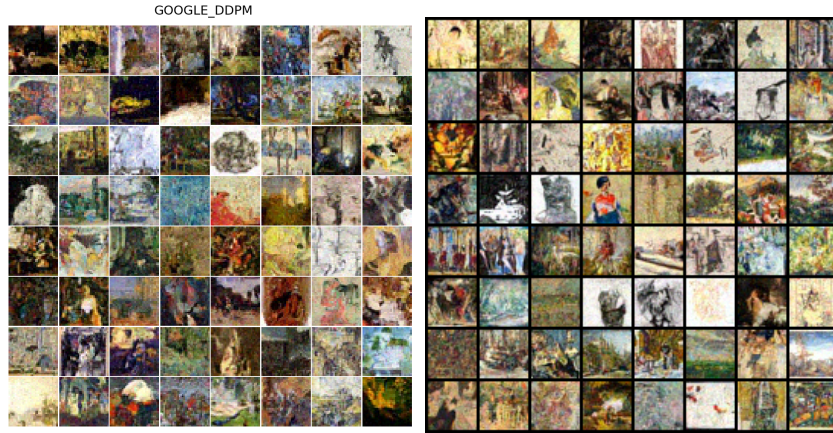


Fig. 3. Samples from the fine-tuned Google DDPM (left: 20 epochs / right: 200 epochs), demonstrating more recognizable shapes compared to the other models

5.2 Quantitative Results

Table 3. Quantitative results (Mean $\pm$ Std Dev) across 10 random seeds for the 20% subset and the full dataset (100%).

Model	20% Subset		Full Dataset		
	FID $\downarrow$	KID ($\times 10^{-3}$) $\downarrow$	FID $\downarrow$	KID ($\times 10^{-3}$) $\downarrow$	IS $\uparrow$
VAE	347.52 ± 0.73	336.1 ± 1.4	355.26 ± 1.32	370.3 ± 4.0	1.90 ± 0.05
DCGAN	121.87 ± 0.70	90.9 ± 1.2	91.02 ± 0.65	69.0 ± 1.5	3.15 ± 0.10
cGAN	301.88 ± 0.81	258.4 ± 1.7	393.68 ± 1.36	329.1 ± 2.6	2.10 ± 0.04
Diffusion	176.58 ± 1.64	120.7 ± 2.8	43.03 ± 0.51	26.7 ± 1.0	4.00 ± 0.16
Google DDPM	122.82 ± 1.30	121.3 ± 2.5	97.49 ± 1.26	90.5 ± 1.5	2.89 ± 0.08

Table 3 presents the results of the 10-seed evaluation on both the full dataset and the 20% subset used for hyperparameter selection. On the full dataset, FID and KID rankings are in perfect agreement across all five models, and per-seed standard deviations are small relative to mean values (e.g., FID std of 0.51 for the UNet, 0.65 for DCGAN), confirming the stability of both the models and the evaluation protocol.

The Inception Score (IS) [9] measures two complementary properties of a generative model’s output distribution: *objectness* (each generated image should be confidently classifiable into some category, i.e., the conditional label distribution $p(y|x)$ should have low entropy) and *diversity* (the marginal $p(y)$ should be spread across many categories, i.e., high entropy). IS is computed on fake images only, using a pretrained Inception V3 classifier [10], and is therefore independent of the real data distribution.

The IS ranking (Table ??) largely agrees with FID/KID. The VAE’s low score reflects the blurry outputs that the Inception classifier cannot assign to any style. The cGAN scores only marginally above the VAE despite its conditioning mechanism, consistent with its classwise mode collapse producing outputs that are coherent but not diverse. Note that IS scores on 32×32 images are inherently lower than those reported on higher-resolution benchmarks (e.g., ImageNet), since the Inception classifier’s softmax confidences decrease with smaller inputs.

On the 20% subset, the ranking shifts notably: DCGAN achieves the best KID ($90.9 \pm 1.2 \times 10^{-3}$) and the best FID (121.87), with Google DDPM close behind (FID = 122.82). Diffusion (UNet) ranks third by FID (176.58) despite dominating on the full dataset, suggesting that data volume is a stronger determinant of diffusion model quality than for GANs. The ordering of VAE and cGAN remains poor in both settings, though cGAN shows a notable relative improvement on the subset (FID = 301.88 vs. 393.68 on the full dataset), which indicates less severe mode collapse when the per-class data distribution is smaller.

5.3 Discussion

Key Trade-offs. Our results highlight several relations between models:

- **Unconditional vs. Conditional:** Despite having access to class labels, the cGAN significantly underperformed the unconditional DCGAN. As analyzed in depth below, it converges to a per-class fixed point that superficially fools the discriminator without covering true intra-class diversity.
- **Diffusion Dominance:** On the full dataset, diffusion models achieved the best FID by a large margin, benefiting from a stable MSE loss landscape and skip connections. On the 20% subset, however, it lost to other models, indicating they require more data to realize their generative advantage. Visually, they exhibit a wide diversity of styles, representing landscapes, figures, line drawings, and abstract pieces with convincing low-level texture throughout.
- **Perceptual Quality:** While UNet Diffusion achieved the lowest global FID, the fine-tuned Google DDPM produced the most coherent samples by visual inspection despite a higher FID. This is because the CIFAR-10 pretraining encodes photographic structure that persists after fine-tuning, and this stylistic difference from ArtBench’s painting distribution penalized by FID/KID. Additionally, despite good FID and producing samples with visible texture and good color diversity, DCGAN samples suffer from checkerboard noise, worsening its perceived quality. These findings highlight that metrics do not always align with human perception of quality.
- **VAE smoothness:** VAEs are optimized with an L_2 -style reconstruction loss optimized for the mean: when multiple high-frequency outcomes are plausible, the decoder is rewarded for predicting an average, which tends to wash out texture and contrast. This effect belongs to the model/loss combination, and helps explain the extremely smooth samples, that loosely

capture the global brownish color palette of the dataset but entirely lack fine detail.

Inception Score as a Complementary Metric. FID and KID penalize both low fidelity and low coverage but cannot separate the two failure modes. For 32×32 images, IS is especially complementary because FID/KID operate on upsampled 299×299 features, introducing interpolation artifacts. IS operates on the classifier’s final softmax, which is more robust to resolution. A model that achieves low FID by generating blurry averages will be penalized by IS (low objectness), while a model with high IS but low diversity will be penalized by FID/KID. Together the three metrics should provide a comprehensive assessment of sample quality, diversity, and distributional fidelity from different angles.

VAE Latent Space Analysis. To investigate the structure learned by the VAE encoder, we trained two variants and visualized their latent spaces via UMAP [7] using 5,000 encoded training images (Figure 4). The first variant uses $\beta = 1$ (standard VAE), where we expected the KL regularizer to align the posterior with a Gaussian prior. In contrast, the second uses $\beta = 0$, effectively turning the model into a deterministic autoencoder with no regularization pressure on the latent space. Without any KL penalty, the encoder is free to allocate latent dimensions arbitrarily, so we expected it to exploit the extra freedom to push each class into a distinct, tight cluster.

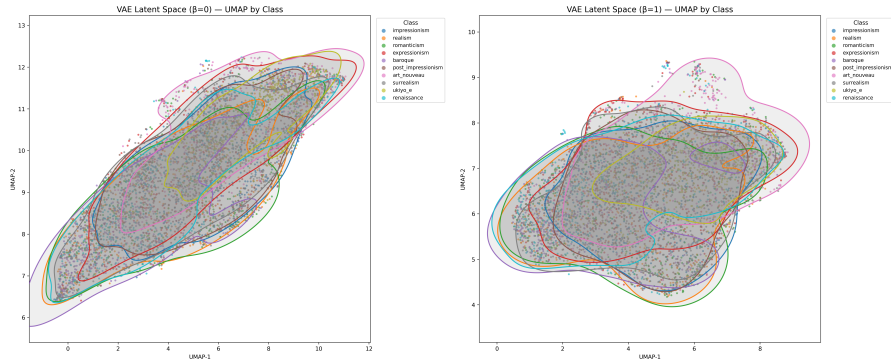


Fig. 4. UMAP projections of the VAE latent space (5,000 samples) for $\beta = 0$ and $\beta = 1$, using KDE soft boundaries. Neither configuration produces clearly separated class clusters, suggesting the bottleneck discards class-discriminative information.

Both visualizations turned out to be relatively unstructured, with no clearly separable class clusters in either setting. We hypothesize that the core issue lies in the information bottleneck being too narrow for the complexity of ArtBench-10. A 32×32 painting contains fine stylistic cues that are specific to each artistic style. Compressing this into a low-dimensional bottleneck forces the encoder to prioritize low-frequency shape and global color, which are largely shared across

styles, at the expense of the class-discriminative high-frequency details. The latent vectors therefore all look similar to UMAP regardless of the class label: the encoder has only retained the information that is common to all classes.

cGAN Class-Wise Mode Collapse. To diagnose the cGAN’s failure mode, we trained two configurations and visualized the discriminator’s internal feature space via UMAP (Figure 5). Features are extracted from the penultimate layer of the discriminator, which receives both the image and its class label as input, so the representation captures what the discriminator considers realistic.

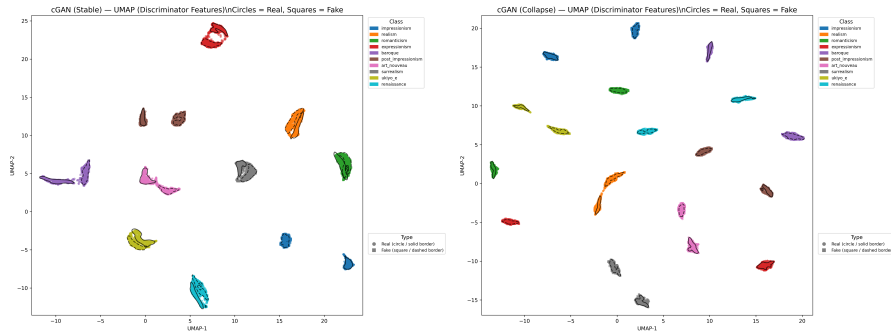


Fig. 5. UMAP of discriminator features (5,000 real + 5,000 fake samples) for the stable and collapsed cGAN. Solid KDE border = real cluster, dashed = fake cluster. In the stable run, fake borders spread wider and overlap more with their real counterparts, indicating higher intra-class diversity. In the collapsed run, fake clusters are degenerate and misaligned with the real distribution.

The **collapsed** configuration (`d_updates_per_g`=1, mismatch weight $\lambda = 0.2$) shows tightly packed fake clusters that do not spread across the real distribution: the generator has found a per-class fixed point that superficially fools the discriminator without covering the true intra-class diversity.

The **stable** configuration (`d_updates_per_g`=5, $\lambda = 1.0$) shows substantially higher cluster-wise variance in the fake distribution: the dashed KDE borders spread further and overlap more with the corresponding real borders, indicating the generator has been pushed to explore the full intra-class diversity rather than collapsing to a single mode. Two complementary mechanisms drove this improvement. First, increasing `d_updates_per_g` to 5 gives the discriminator more signal per generator step, making it harder for the generator to find and exploit a degenerate fixed point. Second, raising the mismatch loss weight λ to 1.0 penalizes the discriminator heavily for accepting real images paired with wrong labels, forcing it to learn genuinely class-discriminative features rather than settling for general realism cues, and by extension forcing the generator to produce images that are class-specifically realistic.

DCGAN Interpolation and Diffusion Denoising. Figure 6 illustrates two complementary model properties. The DCGAN latent space supports smooth interpolation: linearly traversing between two random noise vectors produces a coherent sequence of paintings with gradual style transitions, indicating that the generator has learned a well-structured latent manifold. The diffusion model’s denoising chain shows the reverse process explicitly: pure Gaussian noise is progressively refined over many steps until a recognizable painting emerges, confirming that the model has learned to incrementally remove noise in a semantically meaningful way.

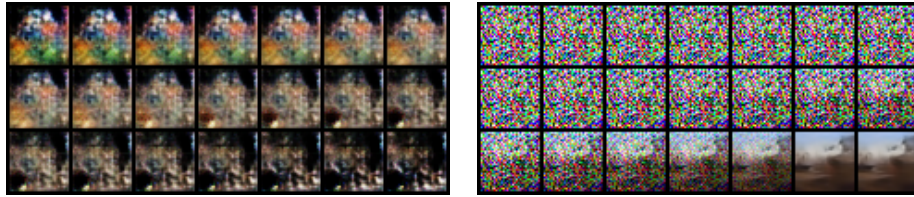


Fig. 6. Left: Linear interpolation between two DCGAN latent vectors (10 steps), showing smooth perceptual transitions. Right: Diffusion denoising chain (10 evenly-spaced steps), from pure Gaussian noise to a generated painting.

6 Conclusion

Across ArtBench-10, the three generative families expose distinct trade-offs between training stability, data efficiency, and perceptual quality. The custom UNet diffusion model achieved the best quantitative results on the full dataset (FID=43.03, IS=4.00), but required full data volume to do so. The DCGAN generalized well with less data, while its conditional variant consistently underperformed across all configurations due to the complexity of the training objective for this data amount and the VAE produced systematically blurred outputs due to its L_2 reconstruction objective. The fine-tuned Google DDPM, despite weaker FID, produced the most visually coherent samples, displaying gaps between distribution metrics and human perception on artistic domains.

References

1. Bińkowski, M., Sutherland, D.J., Arbel, M., Gretton, A.: Demystifying MMD gans. In: Proceedings of the 6th International Conference on Learning Representations (ICLR) (2018)
2. Google: DDPM CIFAR-10 32x32. <https://huggingface.co/google/ddpm-cifar10-32> (2020)

Note on AI Usage: Gemini and Claude assisted in LaTeX formatting and analysis verification.

3. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: Gans trained by a two time-scale update rule converge to a local nash equilibrium. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 30 (2017)
4. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 33, pp. 6840–6851 (2020)
5. Kingma, D.P., Welling, M.: Auto-encoding variational bayes. In: *Proceedings of the 2nd International Conference on Learning Representations (ICLR)* (2014)
6. Liao, P., Li, X., Liu, X., Keutzer, K.: The artbench dataset: Benchmarking generative models with artworks (2022)
7. McInnes, L., Healy, J., Melville, J.: UMAP: Uniform manifold approximation and projection for dimension reduction (2018)
8. Radford, A., Metz, L., Chintala, S.: Unsupervised representation learning with deep convolutional generative adversarial networks. In: *Proceedings of the 4th International Conference on Learning Representations (ICLR)* (2016)
9. Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., Chen, X.: Improved techniques for training gans. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 29 (2016)
10. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 2818–2826 (2016)